



Stutengarten2026

Backend – IT-Dokumentation

Bank-Simulation für Auszubildende Fachinformatiker (Jahrgang 2024)

Technische Referenz für Architektur, Datenmodell, Business-Logik und REST-API

Repository: LBBW-Azubis/Stutengarten2026

Verfasser: Julian Jesinger, Jannik Klys

Stand: Juli 2026

Inhaltsverzeichnis

Inhaltsverzeichnis	2
1. Einleitung	4
2. Architektur & Technologie-Stack	4
2.1 Überblick	4
2.2 Technologie-Stack	4
2.3 Projektstruktur (Backend)	5
3. Datenbankschema	5
3.1 kunden	5
3.2 kundensparbuecher	5
3.3 kundenumsaetze	6
3.4 aktien	6
3.5 kundenaktien	6
3.6 unternehmen	6
3.7 unternehmenssparbuecher	7
3.8 unternehmensumsaetze	7
3.9 kundenstatistik / 3.10 unternehmensstatistik	7
3.11 wirtschaftsbeihilfe	7
4. Datenzugriffsschicht: db_connector.py	8
4.1 Wichtige Konstanten	8
4.2 Methoden	8
5. Business-Logik-Klassen	9
5.1 customer.py – Customer	9
5.2 customer_saving_book.py – CustomerSavingsBook	9
5.3 customer_transactions.py – CustomerTransaction	10
5.4 company.py – Company	10
5.5 company_saving_book.py – CompanySavingsBook	11
5.6 company_transactions.py – CompanyTransaction	11
5.7 share.py – Share	12
5.8 customer_shares.py – CustomerShares	12
5.9 xlsx_file_reader.py – XlsxFileReader	12
5.10 customer_import.py – CustomerImport	13
5.11 company_import.py – CompanyImport	13
6. Fehlerbehandlung / Exception-Hierarchie	14
7. REST-API (Flask-Blueprints)	14
7.1 health_bp – /ping, /health, /health/pool, /clear-database	14
7.2 imports_bp – Massenimport	15
7.3 customers_bp – Kundenverwaltung	15
7.4 companies_bp – Unternehmensverwaltung	16

7.5 shares_bp – Aktien-Stammdaten	18
7.6 settings_bp – Anwendungseinstellungen	18
8. Sicherheit	18
9. Bekannte Probleme & offene Punkte	19
9.1 Verbindungs-Leck bei Company-Neuanlage	19
9.2 Aktien-Symbol-Kollisionen	19
9.3 Blueprint-Registrierung	19
9.4 wirtschaftsbeihilfe ungenutzt	19
9.5 Saldo-Prüfung nur bei übergebenem Sparbuch-Objekt	19
10. Konfiguration & Deployment	20
10.1 server.config	20
10.2 Start der Anwendung (app.py)	20
10.3 Windows-Dienst (NSSM)	20
10.4 Datenbank-Setup	20
11. Zusammenfassung	20

1. Einleitung

Dieses Dokument beschreibt das Backend der Anwendung „Stutengarten2026“ – eine Banksimulation, die im Rahmen der Ausbildung von Fachinformatikern (Jahrgang 2024) eingesetzt wird. Kinder und Unternehmen erhalten virtuelle Konten (Sparbücher), können Geld einzahlen, auszahlen, überweisen und Aktien handeln. Ziel dieses Dokuments ist es, außenstehenden Entwicklern einen vollständigen Überblick über den Backend-Code zu geben: welche Komponenten es gibt, wie sie zusammenspielen, welche Datenbanktabellen existieren und über welche REST-Endpunkte das Frontend mit dem Backend kommuniziert.

Das Frontend (React, siehe eigenes Repository-Verzeichnis) ist nicht Bestandteil dieser Dokumentation und wird nur insoweit erwähnt, wie es zum Verständnis der API-Verträge notwendig ist.

2. Architektur & Technologie-Stack

2.1 Überblick

Das Backend ist eine klassische mehrschichtige REST-API-Anwendung auf Basis von Flask. Es lässt sich grob in vier Schichten gliedern:

- API-Schicht (Blueprints): Nimmt HTTP-Requests entgegen, validiert Eingaben grob, ruft die Business-Logik auf und formt die JSON-Antwort.
- Business-Logik-Schicht (Active-Record-Klassen): Kapselt fachliche Regeln (z. B. „Kontostand darf nicht negativ werden“) und die zugehörigen SQL-Zugriffe.
- Datenzugriffsschicht (DbConnector): Verwaltet einen Connection-Pool zur MariaDB-Datenbank.
- Datenbank (MariaDB): Persistiert Kunden, Unternehmen, Sparbücher, Umsätze, Aktien und Statistiken.

Anfrage-Fluss: Client (React-Frontend) → Flask-Route (Blueprint) → Business-Logik-Klasse → DbConnector → MariaDB → JSON-Antwort zurück an den Client.

2.2 Technologie-Stack

Methode / Funktion	Zweck	Wirft / Rückgabe
Python 3.11 / Flask	Web-Framework, organisiert über Blueprints (siehe Kapitel 5)	–
Waitress	Produktiver WSGI-Server, mit dem app.py die Flask-App ausliefert (statt Flask-Devserver)	–
MariaDB	Relationale Datenbank; Zugriff über mysql-connector-python	–
mysql-connector-python (pooling)	Verbindungs-Pool (10 Verbindungen), siehe db_connector.py	–
pandas / openpyxl	Lesen von XLSX-Dateien für den Massenimport von Kunden/Unternehmen	–
flask-cors	Erlaubt Cross-Origin-Requests vom React-Frontend	–
NSSM	Registriert app.py als Windows-Dienst (Autostart), produktives Deployment	–

2.3 Projektstruktur (Backend)

Die wichtigsten Dateien im Backend-Verzeichnis:

- app.py – Einstiegspunkt: liest server.config, baut den DB-Pool auf, registriert alle Blueprints, startet Waitress.
- db_connector.py – Connection-Pool-Wrapper für MariaDB.
- customer.py, company.py – Stammdaten-Klassen (Active Record) für Kunden bzw. Unternehmen.
- customer_saving_book.py, company_saving_book.py – Sparbuch-Verwaltung (Kontostände).
- customer_transactions.py, company_transactions.py – Buchungen (Umsätze) inkl. Wochentags-Statistik.
- share.py, customer_shares.py – Aktien (Stammdaten) und Aktienbestände der Kunden (Kauf/Verkauf/Kursänderung).
- xlsx_file_reader.py, customer_import.py, company_import.py – Massenimport aus Excel-Dateien.
- blueprints/*_bp.py – sechs Flask-Blueprints, die die REST-Endpunkte definieren (health, imports, customers, companies, shares, settings).
- server.config – IP, Port, DB-Zugangsdaten (INI-Format, von configparser gelesen).
- database.sql, fill_database.sql, clear_database.sql – Schema-Definition, Testdaten, Reset-Skript.

3. Datenbankschema

Die Datenbank besteht aus 11 Tabellen (siehe database.sql). Fremdschlüssel sind überwiegend mit ON DELETE CASCADE versehen, damit z. B. beim Löschen eines Kunden automatisch Sparbuch, Umsätze und Aktienbestände mitgelöscht werden. Für alle Fremdschlüssel existieren zusätzlich explizite Indizes (Performance-Optimierung für Joins).

3.1 kunden

Stammdaten der Kinder-Konten. „stutengarten_id“ ist die im Frontend sichtbare Kontonummer (z. B. „K1“) und der fachliche Schlüssel, über den fast alle anderen Tabellen verknüpfen.

Spalte	Typ	Constraint	Beschreibung
id	INT	PK, AUTO_INCREMENT	Interner technischer Schlüssel
stutengarten_id	VARCHAR(50)	UNIQUE, NOT NULL	Fachliche Kontonummer, wird in der API als Pfadparameter genutzt
vorname	VARCHAR(50)	NOT NULL	Vorname des Kindes
nachname	VARCHAR(50)	NOT NULL	Nachname des Kindes

3.2 kundensparbuecher

Ein Sparbuch pro Kunde (1:1), wird beim Anlegen eines Kunden automatisch mit Saldo 0 erzeugt.

Spalte	Typ	Constraint	Beschreibung
id	INT	PK, AUTO_INCREMENT	Technischer Schlüssel, wird für Transaktionen referenziert
kunden_fk	VARCHAR(50)	FK → kunden.stutengarten_id, ON DELETE CASCADE	Zugehöriger Kunde
saldo	INT	DEFAULT 0	Aktueller Kontostand in „Stuggis“ (Spielwährung)

3.3 kundenumsaetze

Jede Buchung (Einzahlung, Auszahlung, Überweisung) auf einem Kunden-Sparbuch.

Spalte	Typ	Constraint	Beschreibung
id	INT	PK, AUTO_INCREMENT	–
kundensparbuch_fk	INT	FK → kundensparbuecher.id, ON DELETE CASCADE	Betroffenes Sparbuch
betrag	INT	NOT NULL	Buchungsbetrag, kann negativ sein (Auszahlung)
verwendungszweck	VARCHAR(255)	–	Freitext, z. B. „Einzahlung“, „Überweisung“

3.4 aktien

Katalog aller handelbaren Aktien. Wird sowohl manuell (POST /share) als auch automatisch beim Unternehmensimport befüllt, da jedes Unternehmen zugleich als Aktie handelbar ist.

Spalte	Typ	Constraint	Beschreibung
id	INT	PK, AUTO_INCREMENT	–
name	VARCHAR(100)	UNIQUE, NOT NULL	Anzeigenname der Aktie (i. d. R. Unternehmensname)
symbol	VARCHAR(5)	UNIQUE, NOT NULL	Kürzel, z. B. „APF“
beschreibung	TEXT	–	Freitext-Beschreibung
erstellt_am	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP	Anlagedatum

3.5 kundenaktien

Aktienbestand eines Kunden: pro Kauf ein Datensatz mit investiertem Betrag und aktuellem (durch das „Aktien-Drehrad“ veränderbarem) Wert.

Spalte	Typ	Constraint	Beschreibung
id	INT	PK, AUTO_INCREMENT	–
besitzer_fk	VARCHAR(50)	FK → kunden.stutengarten_id, ON DELETE CASCADE	Besitzender Kunde
aktie_fk	INT	FK → aktien.id, ON DELETE CASCADE	Gekaufte Aktie
investierter_betrag	INT	NOT NULL	Ursprünglich investierter Betrag
aktueller_wert	INT	NOT NULL	Aktueller Wert (wird über das Drehrad angepasst)
wochentage	ENUM(7 Werte)	NOT NULL	Wochentag des Kaufs (deutsche Großschreibung, z. B. „MONTAG“)

3.6 unternehmen

Spalte	Typ	Constraint	Beschreibung
id	INT	PK, AUTO_INCREMENT	–
bezeichnung	VARCHAR(255)	UNIQUE, NOT NULL	Firmenname, fachlicher Schlüssel in der API
mappe_abgegeben	BOOL	DEFAULT FALSE	TRUE = Mappe beim Kunden, FALSE = Mappe bei der Bank

3.7 unternehmenssparbuecher

Spalte	Typ	Constraint	Beschreibung
id	INT	PK, AUTO_INCREMENT	–
unternehmen_fk	INT	FK → unternehmen.id, ON DELETE CASCADE	Zugehöriges Unternehmen
saldo	INT	DEFAULT 0	Kontostand

3.8 unternehmensumsaetze

Spalte	Typ	Constraint	Beschreibung
id	INT	PK, AUTO_INCREMENT	–
unternehmenssparbuecher_fk	INT	FK → unternehmenssparbuecher.id, ON DELETE CASCADE	Betroffenes Sparbuch
betrag	INT	NOT NULL	Buchungsbetrag (positiv/negativ)
verwendungszweck	VARCHAR(255)	–	Freitext

3.9 kundenstatistik / 3.10 unternehmensstatistik

Beide Tabellen kumulieren Umsätze pro Wochentag (für die Balkendiagramme im Frontend, z. B. „Unternehmen Umsatz“). Pro Kunde/Unternehmen und Wochentag existiert höchstens ein Datensatz, der bei jeder neuen Buchung aufaddiert wird.

Spalte	Typ	Constraint	Beschreibung
id	INT	PK, AUTO_INCREMENT	–
kunde_fk / unternehmen_fk	INT	FK, ON DELETE CASCADE	Bezug auf kunden.id bzw. unternehmen.id (technischer PK, nicht stutengarten_id!)
wochentage	ENUM(7 Werte)	NOT NULL	Wochentag
gesamtumsatz	INT	–	Aufsummierter Betrag für diesen Wochentag

3.11 wirtschaftsbeihilfe

In den Backend-Endpunkten aktuell nicht angebunden (kein Blueprint verwendet diese Tabelle); vorbereitet für künftige Funktionalität „Wirtschaftsbeihilfe pro Beruf“.

Spalte	Typ	Constraint	Beschreibung
id	INT	PK, AUTO_INCREMENT	–
beruf	VARCHAR(255)	NOT NULL	Berufsbezeichnung
betrag	INT	NOT NULL	Höhe der Beihilfe
unternehmen_fk	INT	FK → unternehmen.id, ON DELETE CASCADE	Zahlendes Unternehmen

4. Datenzugriffsschicht: db_connector.py

Die Klasse DbConnector kapselt einen `mysql.connector.pooling.MySQLConnectionPool`. Es gibt pro Anwendungsinstanz genau einen Pool (`POOL_NAME = "stutengarten_pool"`) mit 10 Verbindungen (`POOL_SIZE`). Alle Business-Logik-Klassen erhalten diesen Connector über `app.config["DB_CONNECTOR"]` und rufen `db.get_connection()` auf, um eine Verbindung „auszuleihen“.

4.1 Wichtige Konstanten

Methode / Funktion	Zweck	Wirft / Rückgabe
<code>CONNECTION_TIMEOUT = 1s</code>	Timeout für den Aufbau einer neuen TCP-Verbindung zur DB	–
<code>AUTOCOMMIT = False</code>	Transaktionen müssen explizit per <code>conn.commit()</code> bestätigt werden	–
<code>GET_CONN_MAX_WAIT_S = 5.0</code>	Maximale Wartezeit, wenn der Pool gerade voll ausgelastet ist	–
<code>WARN_THRESHOLD_S = 0.3</code>	Ab dieser Wartezeit wird eine Warnung geloggt (Performance-Monitoring)	–

4.2 Methoden

Methode / Funktion	Zweck	Wirft / Rückgabe
<code>connect(host, user, password, database, port)</code>	Initialisiert den Connection-Pool einmalig beim App-Start (wird in <code>app.py</code> aufgerufen)	wirft <code>mysql.connector.Error</code> bei Fehlkonfiguration
<code>get_connection()</code>	Leiht eine Verbindung aus dem Pool; wartet bei vollem Pool bis <code>GET_CONN_MAX_WAIT_S</code> , loggt eine Warnung bei langer Wartezeit und hängt die gemessene Wartezeit als <code>_wait_time_s</code> an das Connection-Objekt	gibt eine <code>mysql.connector</code> -Verbindung zurück; wirft <code>AcquireTimeout</code> bei Zeitüberschreitung
<code>close()</code>	Setzt die Pool-Referenz auf <code>None</code> (Freigabe der Ressourcen beim App-Shutdown)	–

Konsequenz für alle anderen Module: Jede Methode, die auf die Datenbank zugreift, folgt demselben Muster „Connection holen → Cursor öffnen → try/except/finally → Cursor und Connection schließen“. Damit Verbindungen zuverlässig an den Pool zurückgegeben werden, MUSS `conn.close()` in jedem Pfad (auch im Fehlerfall) aufgerufen werden – siehe Hinweis zu einem bekannten Ausnahmefall in Kapitel 9.

5. Business-Logik-Klassen

Alle Business-Logik-Klassen folgen dem Active-Record-Muster: Eine Instanz repräsentiert eine Zeile einer Tabelle bzw. eine fachliche Operation, und die Klasse selbst enthält sowohl die Datenhaltung (Attribute) als auch die SQL-Zugriffe (Methoden). Für JSON-Serialisierung besitzt praktisch jede Klasse eine `to_dict()`-Methode.

5.1 customer.py – Customer

Repräsentiert einen Kunden (Kind). Der Konstruktor arbeitet in zwei Modi: Wird `id` nicht angegeben, wird ein neuer Kunde inklusive automatisch angelegtem Sparbuch in der Datenbank erzeugt (zwei INSERTs in derselben Transaktion). Wird `id` angegeben, wird nur ein In-Memory-Objekt aus bereits geladenen Daten gebaut (kein DB-Zugriff).

Methode / Funktion	Zweck	Wirft / Rückgabe
<code>__init__(db, stutengarten_id, first_name, last_name, id=None)</code>	Neuanlage (<code>id=None</code>) oder Objekt-Konstruktion aus geladenen Daten (<code>id</code> gesetzt). Legt bei Neuanlage automatisch ein Sparbuch mit Saldo 0 an	wirft <code>CustomerException</code> bei fehlenden Pflichtfeldern, <code>CustomException</code> bei DB-Fehlern
<code>get_by_stutengarten_id(db, stutengarten_id) [static]</code>	Lädt einen Kunden anhand der fachlichen Kontonummer	wirft <code>CustomerException</code> , wenn kein Kunde gefunden wurde
<code>get_by_db_id(db, customer_id) [static]</code>	Lädt einen Kunden anhand des technischen Primärschlüssels	wirft <code>CustomerException</code>
<code>get_savings_book()</code>	Liefert eine leichte <code>SavingsBookRef</code> (<code>customer_id</code> , <code>balance</code>) für Guthaben-Prüfungen	wirft <code>CustomException</code> , falls kein Sparbuch existiert
<code>update_stutengarten_id / update_first_name / update_last_name</code>	Ändert einzelne Felder direkt in der DB und im Objekt	–
<code>delete()</code>	Löscht den Kunden (CASCADE löscht Sparbuch, Umsätze, Aktien mit)	–
<code>to_dict()</code>	JSON-taugliches Dictionary mit <code>id</code> , <code>stutengarten_id</code> , <code>first_name</code> , <code>last_name</code>	–

5.2 customer_saving_book.py – CustomerSavingsBook

Reine Utility-Klasse (nur statische Methoden) für Lese-/Schreibzugriffe auf kundensparbuecher, im Gegensatz zu `Customer` nicht als Objekt instanziiert. Arbeitet konsequent mit Dictionaries statt eigenen Objekten, um die JSON-Ausgabe zu vereinfachen.

Methode / Funktion	Zweck	Wirft / Rückgabe
<code>get_all_savings_books(db) [static]</code>	Liste aller Sparbücher mit Kundendaten (Join kundensparbuecher ⋈ kunden)	Liste von Dicts
<code>get_savings_book_overview(db, stutengarten_id) [static]</code>	Sparbuch-Übersicht (Saldo + ID) für einen Kunden	Liste von Dicts
<code>create_new(db, stutengarten_id) [static]</code>	Legt manuell ein neues Sparbuch an (Saldo 0)	wirft <code>CustomException</code> bei Duplikat oder unbekanntem Kunden
<code>set_balance(db, stutengarten_id, balance) [static]</code>	Setzt den Saldo direkt auf einen Wert (nicht additiv!)	wirft <code>CustomException</code> / <code>CustomerException</code> bei ungültigem Betrag oder unbekanntem Kunden

5.3 customer_transactions.py – CustomerTransaction

Bildet einzelne Buchungen ab und aktualisiert dabei atomar den Sparbuch-Saldo sowie eine Wochentags-Statistik (kundenstatistik). Für die deutschen Wochentagsnamen wird versucht, das System-Locale auf de_DE.UTF-8 bzw. German_Germany.1252 zu setzen; schlägt dies fehl, wird eine Warnung ausgegeben, die Wochentagsnamen selbst sind aber hart codiert (['MONTAG', ..., 'SONNTAG'], abhängig von Python-Wochentagsindex, nicht vom Locale) und funktionieren daher auch ohne korrektes Locale.

Methode / Funktion	Zweck	Wirft / Rückgabe
<code>__init__(db, customer_savings_book_id, amount, purpose, customer_savings_book=None, transaction_id=None)</code>	Neuanlage: prüft bei übergebenem <code>customer_savings_book</code> , ob der Saldo nicht unter 0 fällt, schreibt den Umsatz, aktualisiert den Saldo und ruft anschließend <code>add_to_statistics()</code> auf. Alles innerhalb einer Transaktion (commit/rollback)	wirft <code>CustomerException</code> bei negativem Saldo, <code>CustomException</code> bei sonstigen DB-Fehlern
<code>get_all_transactions_for_customer(db, stutengarten_id) [static]</code>	Alle Umsätze eines Kunden, neueste zuerst (Join über 3 Tabellen)	Liste von Dicts inkl. <code>customer_name</code>
<code>add_to_statistics()</code>	Erhöht/erstellt den Tageswert in kundenstatistik für den aktuellen Wochentag	fängt Fehler intern ab und loggt sie nur (kein Rollback der Haupttransaktion)
<code>get_customer_statistics(db, kunde_fk=None) [static]</code>	Summiert gesamtumsatz je Wochentag – für einen Kunden oder global (<code>kunde_fk=None</code>)	Liste von {weekday, total_amount}
<code>transfer(db, from_stutengarten_id, to_stutengarten_id, amount, purpose='') [static]</code>	Überweisung zwischen zwei Kunden: sperrt beide Sparbuch-Zeilen mit <code>SELECT ... FOR UPDATE</code> , prüft Guthaben, bucht auf beiden Seiten je einen Umsatz und aktualisiert die Statistik beider Kunden – alles in einer Transaktion	wirft <code>CustomerTransactionException</code> bei fehlendem Sparbuch oder unzureichendem Guthaben
<code>to_dict()</code>	JSON-Dict mit <code>id</code> , <code>customer_savings_book_id</code> , <code>amount</code> , <code>purpose</code> (optional <code>customer_name</code>)	–

Hinweis: transfer() verwendet SELECT ... FOR UPDATE, um Race Conditions bei gleichzeitigen Überweisungen zu verhindern – wichtig, da mehrere Endgeräte parallel auf dieselbe Datenbank zugreifen können.

5.4 company.py – Company

Analog zu Customer, jedoch für Unternehmen. Fachlicher Schlüssel ist hier der eindeutige Firmenname (bezeichnung), nicht eine separate ID wie bei Kunden.

Methode / Funktion	Zweck	Wirft / Rückgabe
<code>__init__(db, name=None, folder_handed_over=False, id=None)</code>	Neuanlage inkl. automatisch angelegtem Unternehmensspargbuch, oder Objekt-Konstruktion aus geladenen Daten	wirft <code>CompanyException</code> bei fehlendem Namen, <code>CustomCompanyException</code> bei DB-Fehlern
<code>get_by_name(db, name) [static]</code>	Lädt ein Unternehmen anhand des Namens	wirft <code>CompanyException</code> , wenn nicht gefunden
<code>get_by_db_id(db, company_id) [static]</code>	Lädt ein Unternehmen anhand der technischen ID	wirft <code>CompanyException</code>
<code>get_savings_book()</code>	Liefert <code>CompanySavingsBookRef</code> (<code>company_id</code> , <code>balance</code>)	wirft <code>CustomCompanyException</code> , falls kein Sparbuch existiert
<code>update_name(new_name) / update_folder_handed_over(bool)</code>	Ändert Namen bzw. Mappe-Status	–
<code>delete()</code>	Löscht das Unternehmen (CASCADE löscht Sparbuch/Umsätze mit)	–
<code>to_dict()</code>	JSON-Dict mit <code>id</code> , <code>name</code> , <code>folder_handed_over</code>	–

Bekannte Auffälligkeit: Im Neuanlage-Zweig von `__init__` wird im finally-Block nur `cursor.close()` aufgerufen, nicht `conn.close()` – siehe Kapitel 9 („Bekannte Probleme“).

5.5 company_saving_book.py – CompanySavingsBook

Pendant zu CustomerSavingsBook für Unternehmen, ebenfalls reine Utility-Klasse mit statischen Methoden.

Methode / Funktion	Zweck	Wirft / Rückgabe
<code>get_all_company_savings_books(db)</code> [static]	Liste aller Unternehmens-Sparbücher inkl. Name und Mappe-Status	Liste von Dicts
<code>get_company_savings_book_overview(db, company_id)</code> [static]	Sparbuch-Übersicht für ein bestimmtes Unternehmen	Liste von Dicts
<code>create_new(db, company_id)</code> [static]	Legt manuell ein neues Unternehmens-Sparbuch an	wirft CustomCompanyException bei DB-Fehler
<code>set_balance(db, company_id, balance)</code> [static]	Setzt den Saldo direkt (nicht additiv)	wirft CustomCompanyException bei ungültigem Betrag oder unbekanntem Unternehmen

5.6 company_transactions.py – CompanyTransaction

Strukturell identisch zu CustomerTransaction, jedoch für Unternehmensumsätze und unternehmensstatistik. Es gibt keine `transfer()`-Methode – Unternehmen können untereinander nicht direkt überweisen; das ist ausschließlich Kunden vorbehalten.

Methode / Funktion	Zweck	Wirft / Rückgabe
<code>__init__(db, company_savings_book_id, amount, purpose, company_savings_book=None, transaction_id=None)</code>	Neuanlage einer Buchung, Saldo-Update und <code>add_to_statistics()</code> – analog zu CustomerTransaction	wirft CustomCompanyException bei negativem Saldo oder DB-Fehlern
<code>get_all_transactions_for_company(db, company_id)</code> [static]	Alle Umsätze eines Unternehmens, neueste zuerst	Liste von Dicts inkl. <code>company_name</code>
<code>add_to_statistics()</code>	Aktualisiert unternehmensstatistik für den aktuellen Wochentag	loggt Fehler intern, kein Rollback der Haupttransaktion
<code>get_company_statistics(db, unternehmen_fk=None)</code> [static]	Wochentags-Summen für ein Unternehmen oder global	Liste von {weekday, total_amount}
<code>to_dict()</code> / <code>get_id()</code> / <code>get_company_savings_book_id()</code> / <code>get_amount()</code> / <code>get_purpose()</code>	JSON-Serialisierung bzw. klassische Getter (Altlasten für Rückwärtskompatibilität)	–

5.7 share.py – Share

Repräsentiert eine handelbare Aktie (Stammdaten, unabhängig vom Bestand einzelner Kunden). Enthält die Hilfsfunktion `_generate_unique_symbol(cursor, base_name)` außerhalb der Klasse, die aus den ersten drei alphanumerischen Zeichen des Namens ein Kürzel bildet, sofern kein Symbol explizit übergeben wurde.

Methode / Funktion	Zweck	Wirft / Rückgabe
<code>__init__(db, share_id=None, name=None, symbol=None, description=None)</code>	Neuanlage (prüft Namens- und Symbol-Eindeutigkeit, generiert bei Bedarf ein Symbol) oder Objekt-Konstruktion aus geladenen Daten	wirft <code>ShareException</code> bei Namens-/Symbol-Kollision oder fehlendem Namen
<code>get_all_available_shares(db) [static]</code>	Alle Aktien, alphabetisch sortiert	Liste von Share-Objekten
<code>get_by_id(db, share_id) [static]</code>	Lädt eine Aktie über die ID	wirft <code>ShareException</code> , wenn nicht gefunden
<code>get_by_name(db, name) [static]</code>	Lädt eine Aktie über den Namen (wird häufig vom Frontend genutzt, da Kunden Aktien per Namen kaufen)	wirft <code>ShareException</code>
<code>to_dict()</code>	JSON-Dict mit id, name, symbol, description	–

Bekannte Einschränkung: `_generate_unique_symbol` nutzt nur die ersten drei Zeichen des Namens als Kürzel und wirft bei Kollision sofort eine `ShareException`, statt automatisch einen numerischen Suffix anzuhängen (Kürzel-Feld erlaubt bis zu 5 Zeichen). Bei vielen Firmen mit ähnlichem Anfangsbuchstaben führt das zu Fehlschlägen beim Import (siehe Kapitel 9).

5.8 customer_shares.py – CustomerShares

Bildet den Aktienbestand eines Kunden ab: Kauf, Verkauf, manuelle Kursänderung („Aktien-Drehrad“) und Gewinn-/Verlust-Berechnung.

Methode / Funktion	Zweck	Wirft / Rückgabe
<code>__init__(db, share_id, customer_stutengarten_id, invested_amount, customer_shares_id=None, customer_savings_book=None)</code>	Neukauf: prüft, ob der Kunde die Aktie bereits besitzt und ob genug Guthaben vorhanden ist, bucht den investierten Betrag vom Spargbuch ab und legt den Bestand an. Bei gesetztem <code>customer_shares_id</code> nur Objektaufbau (Laden)	wirft <code>CustomerException</code> bei Doppelkauf oder unzureichendem Guthaben
<code>get_all_customer_shares(db, stutengarten_id) [static]</code>	Alle Aktien eines Kunden inkl. berechnetem <code>profit_loss</code> und <code>profit_loss_percent</code>	Liste von CustomerShares-Objekten
<code>sell(customer_savings_book=None)</code>	Verkauft die Position zum aktuellen Wert: schreibt den Erlös dem Spargbuch gut und löscht den Bestandsdatensatz	gibt Dict mit <code>invested_amount</code> , <code>sell_value</code> , <code>profit</code> zurück
<code>update_share_value(db, aktie_fk, new_value, stutengarten_id) [static]</code>	„Aktien-Drehrad“: skaliert den Gesamtwert aller Positionen eines Kunden in dieser Aktie proportional auf <code>new_value</code> (Mindestwert 0)	wirft <code>CustomerException</code> , wenn keine Position existiert
<code>to_dict() / get_id() / get_customer_stutengarten_id() / get_value()</code>	JSON-Serialisierung bzw. Getter	–

5.9 xlsx_file_reader.py – XlsxFileReader

Schlanker Wrapper um `pandas.read_excel()`: liest eine hochgeladene XLSX-Datei (Flask FileStorage- oder Stream-Objekt) und liefert die Zeilen als Liste von Dictionaries (`orient="records"`). Versucht zunächst mit der Engine „openpyxl“ zu lesen und fällt bei einem `ValueError` auf die pandas-Standard-Engine zurück.

5.10 customer_import.py – CustomerImport

Massenimport von Kunden aus einer XLSX-Datei. Erkennt flexibel unterschiedliche Spaltenüberschriften (card_id/Stutengarten ID/Stutengarten_ID/ID sowie name/Vorname, surname/Nachname). Zeilen mit fehlenden Pflichtfeldern werden stillschweigend übersprungen.

Methode / Funktion	Zweck	Wirft / Rückgabe
<code>import_customers(file_object=None)</code>	Orchestriert Lesen (XlsxFileReader) und Schreiben (<code>insert_customers</code>); liefert eine JSON-taugliche Erfolgs-/Fehlermeldung	gibt Dict mit status, message, total_records, inserted_records zurück
<code>insert_customers(customer_data)</code>	Bulk-Insert mit INSERT IGNORE INTO kunden (<code>executemany</code>) und anschließend Bulk-Insert der zugehörigen Sparbücher (ebenfalls INSERT IGNORE), alles in einer Transaktion	gibt <code>cursor.rowcount</code> (Anzahl eingefügter Kunden) zurück; wirft die Original-Exception weiter, wenn der Bulk-Insert komplett fehlschlägt

5.11 company_import.py – CompanyImport

Massenimport von Unternehmen. Legt zu jedem importierten Unternehmen zusätzlich automatisch eine Aktie mit gleichem Namen an (Unternehmen sind im Spiel gleichzeitig Aktien) sowie das Unternehmens-Sparbuch.

Methode / Funktion	Zweck	Wirft / Rückgabe
<code>_generate_symbol(name) [static]</code>	Erzeugt ein Aktienkürzel aus den ersten 5 Buchstaben des Namens (nur Buchstaben, Großschreibung), z. B. „Apfel AG“ → „APFLA“	gibt String zurück (keine Kollisionsprüfung!)
<code>import_company(file_object=None)</code>	Orchestriert Lesen und Schreiben, liefert Erfolgs-/Fehlermeldung	Dict mit status, message, total_records, inserted_records
<code>insert_companies(company_data)</code>	Vierstufiger Bulk-Vorgang: (1) INSERT IGNORE unternehmen, (2) INSERT IGNORE aktien, (3) Nachladen der Unternehmens-IDs per SELECT ... IN (...), (4) INSERT IGNORE unternehmenssparbuecher – alles in einer Transaktion	gibt Anzahl neu eingefügter Unternehmen zurück

Wichtiger Unterschied zu share.py: Die hier verwendete `_generate_symbol()`-Funktion prüft keine Kollisionen. Da `unternehmen.name` UNIQUE ist, verhindert INSERT IGNORE zwar doppelte Firmennamen, aber zwei unterschiedlich benannte Firmen mit gleichem 5-Buchstaben-Präfix erzeugen einen Symbol-Konflikt in `aktien.symbol` (ebenfalls UNIQUE); die betreffende Zeile wird dann von INSERT IGNORE stillschweigend übersprungen, ohne dass Firma/Fehler im Ergebnis auftauchen (siehe Kapitel 9).

6. Fehlerbehandlung / Exception-Hierarchie

Jedes fachliche Modul definiert eigene Exception-Klassen, die in den Blueprints gezielt abgefangen und auf passende HTTP-Statuscodes gemappt werden. Es gibt keine gemeinsame Basisklasse – die Modultrennung erfolgt bewusst pro Domäne (Kunde/Unternehmen/Aktie).

Methode / Funktion	Zweck	Wirft / Rückgabe
CustomException, CustomerException (customer.py)	Allgemeiner DB-/Validierungsfehler bzw. fachlicher Regelverstoß (z. B. „Kunde nicht gefunden“, „Saldo würde negativ“)	customers_bp mappt meist auf 400/404/500
CustomerTransactionException (customer_transactions.py)	Fehler bei Buchungen/Überweisungen (fehlendes Sparguthaben, unzureichendes Guthaben)	customers_bp → 400
CustomCompanyException, CompanyException (company.py)	Analog zu Customer-Exceptions, für Unternehmen	companies_bp → 400/404/500
CompanyTransactionException (company_transactions.py)	Fehler bei Unternehmensbuchungen	companies_bp → 400
ShareException (share.py)	Fehler bei Aktien-Stammdaten (Name/Symbol bereits vergeben, nicht gefunden)	shares_bp / customers_bp → 400/404

Alle Blueprint-Routen besitzen zusätzlich einen generischen except Exception-Fallback, der eine 500-Antwort mit Fehlerdetails liefert, damit unerwartete Fehler nicht zu einem unbehandelten Server-Absturz führen, sondern als JSON-Fehlermeldung beim Client ankommen.

7. REST-API (Flask-Blueprints)

Die API ist in sechs Blueprints aufgeteilt, die in app.py registriert werden. Alle Endpunkte erwarten/liefern JSON (außer den beiden Import-Endpunkten, die multipart/form-data mit einer XLSX-Datei erwarten). Es existiert keine Authentifizierung auf API-Ebene – der Zugriffsschutz erfolgt ausschließlich im Frontend (Betreuer-Passwort) sowie durch Beschränkung auf das lokale Netz.

7.1 health_bp – /ping, /health, /health/pool, /clear-database

Dient der Betriebsüberwachung und administrativen Reset-Funktion. TRUNCATE_TABLES enthält alle 11 fachlichen Tabellen in einer Reihenfolge, die Fremdschlüssel-Konflikte vermeidet (FOREIGN_KEY_CHECKS wird zusätzlich temporär deaktiviert).

Methode	Pfad	Beschreibung	Body / Parameter	Antwort
GET	/ping	Sehr schneller Liveness-Check ohne DB-Zugriff	–	200 {ok: true}
GET	/health	Prüft DB-Erreichbarkeit (SELECT 1) und misst die Wartezeit beim Ausleihen einer Connection	–	200/503 {status, database, wait_ms}
GET	/health/pool?n=5	Leiht n Verbindungen nacheinander aus (Lasttest des Pools), liefert min/avg/max Wartezeit	Query: n (1–500)	200/206/503 {status, waits{...}}
POST	/clear-database	Leert alle fachlichen Tabellen (TRUNCATE) und setzt AUTO_INCREMENT zurück – wird vom Betreuer-Menü „Datenbank Löschen“ aufgerufen	–	200 {status, tables} / 500 bei Fehler

7.2 imports_bp – Massenimport

Methode	Pfad	Beschreibung	Body / Parameter	Antwort
POST	/customer/import	Importiert Kunden aus einer XLSX-Datei (Feldname „customers“ oder „file“)	multipart/form-data, Datei .xlsx	200 {status, message, total_records, inserted_records} / 400
POST	/company/import	Importiert Unternehmen aus einer XLSX-Datei inkl. automatischer Aktienanlage	multipart/form-data, Datei .xlsx	200 {status, message, total_records, inserted_records} / 400

7.3 customers_bp – Kundenverwaltung

Methode	Pfad	Beschreibung	Body / Parameter	Antwort
GET	/customer/<stutengarten_id>	Kunde anhand der Kontonummer laden	-	200 Customer / 404
POST	/customer	Neuen Kunden anlegen (inkl. Sparbuch)	{stutengarten_id, first_name, last_name}	200 Customer / 400 / 500
PATCH	/customer/<stutengarten_id>	Einzelne Felder ändern	{stutengarten_id?, first_name?, last_name?}	200 {status, updated, customer} / 404
DELETE	/customer/<stutengarten_id>	Kunde löschen (CASCADE)	-	200 {status, deleted id} / 404
GET	/customersavingsbook	Übersicht aller Kunden-Sparbücher	-	200 [{stutengarten_id, first_name, last_name, balance}]
GET	/customer/<stutengarten_id>/savingsbook	Sparbuch eines Kunden	-	200 [{stutengarten_id, balance}]
PATCH	/customer/<stutengarten_id>/savingsbook/balance	Saldo direkt setzen (nicht additiv!)	{balance}	200 {stutengarten_id, first_name, last_name, balance} / 400
POST	/customer/<stutengarten_id>/transaction	Einzelbuchung (Ein-/Auszahlung); amount negativ = Auszahlung	{amount, purpose}	201 Transaction / 400 / 404
GET	/customer/<stutengarten_id>/transactions	Alle Umsätze eines Kunden	-	200 [Transaction]
GET	/customer/statistics	Wochentags-Statistik über alle Kunden	-	200 [{weekday, total_amount}]
GET	/customer/<stutengarten_id>/statistics	Wochentags-Statistik für einen Kunden	-	200 [{weekday, total_amount}] / 404
POST	/customer/transfer	Überweisung zwischen zwei	{from, to, amount, purpose}	201 {status, from, to, amount} / 400

Methode	Pfad	Beschreibung	Body / Parameter	Antwort
		Kunden (atomar, mit Zeilensperre)		
GET	/shares/available	Alle verfügbaren Aktien (Alias zu shares_bp)	–	200 [Share]
GET	/shares/name/<share_name>	Aktie über den Namen suchen	–	200 Share / 404
POST	/customer/<stutengarten_id>/shares/buy	Aktie kaufen (Betrag frei wählbar)	{share_name, amount}	201 {..., new_balance} / 400
GET	/customer/<stutengarten_id>/shares	Aktienportfolio eines Kunden inkl. Gewinn/Verlust	–	200 [CustomerShares] / 404
PATCH	/customer/<stutengarten_id>/shares/<share_name>	Kurs einer Aktienposition manuell setzen (Drehrad)	{new_value}	200 {status, message, ...} / 404
DELETE	/customer/<stutengarten_id>/shares/<share_name>	Aktienposition(en) verkaufen (alle Positionen dieses Namens)	–	200 {status, invested_amount, sell_value, profit} / 404

7.4 companies_bp – Unternehmensverwaltung

Methode	Pfad	Beschreibung	Body / Parameter	Antwort
GET	/company/<company_name>	Unternehmen anhand des Namens laden	–	200 Company / 404
POST	/company	Neues Unternehmen anlegen (inkl. Sparbuch)	{name, folder_handed_over}	201 Company / 400 / 500
PATCH	/company/<company_name>	Name und/oder Mappe-Status ändern	{name?, folder_handed_over?}	200 {status, updated, company} / 404
DELETE	/company/<company_name>	Unternehmen löschen (CASCADE)	–	200 {status, deleted} / 404
GET	/companysavingsbook	Übersicht aller Unternehmens-Sparbücher	–	200 [{name, folder_handed_over, balance}]
GET	/company/<company_name>/savingsbook	Sparbuch eines Unternehmens	–	200 [{name, balance}] / 404
PATCH	/company/<company_name>/savingsbook/balance	Saldo direkt setzen	{balance}	200 {company_id, name, balance} / 404
GET	/company/<company_name>/transactions	Alle Umsätze eines Unternehmens	–	200 [Transaction] / 404

Methode	Pfad	Beschreibung	Body / Parameter	Antwort
POST	/company/<company_name>/transactions	Einzelbuchung (Ein- /Auszahlung)	{amount, purpose}	201 Transaction / 400 / 404
GET	/company/statistics	Wochentags- Statistik über alle Unternehmen	–	200 [{weekday, total_amount}]
GET	/company/<company_name>/statistics	Wochentags- Statistik für ein Unternehmen	–	200 [{weekday, total_amount}] / 404

7.5 shares_bp – Aktien-Stammdaten

Methode	Pfad	Beschreibung	Body / Parameter	Antwort
POST	/share	Neue Aktie anlegen (Symbol optional, wird sonst generiert)	{name, symbol?, description?}	201 Share / 400
GET	/share/<share_name>	Aktie über den Namen laden	–	200 Share / 404
GET	/shares	Alle Aktien	–	200 [Share]

7.6 settings_bp – Anwendungseinstellungen

Persistiert globale Einstellungen (Hacker-Minispiel, Spiele-Freigabe) threadsicher (per Lock) in einer settings.json-Datei im Backend-Wurzelverzeichnis, damit alle angeschlossenen Clients (mehrere PCs) dieselben Einstellungen sehen. Das Frontend pollt /settings alle 5 Sekunden.

Methode	Pfad	Beschreibung	Body / Parameter	Antwort
GET	/settings	Aktuelle Einstellungen laden (mit Defaults aufgefüllt)	–	200 {hackerAktiv, hackerIntervall, hackerAutoStart, spieleAktiv, spieleAutoStart}
PATCH	/settings	Einzelne Einstellungen aktualisieren (unbekannte Keys werden ignoriert)	{hackerAktiv?, hackerIntervall?, hackerAutoStart?, spieleAktiv?, spieleAutoStart?}	200 vollständiges Settings-Objekt / 400

8. Sicherheit

- Alle SQL-Statements verwenden parametrisierte Queries (%s-Platzhalter über mysql-connector), keine String-Konkatenation → Schutz vor SQL-Injection.
- CORS ist global über flask_cors.CORS(app) aktiviert, damit das React-Frontend (anderer Origin/Port) zugreifen darf.
- Es gibt keine Authentifizierung/Autorisierung auf API-Ebene. Der einzige Zugriffsschutz ist das Betreuer-Passwort im Frontend (client-seitig, nicht durch das Backend erzwungen) sowie die Beschränkung auf das lokale Schulnetz.
- server.config enthält Datenbank-Zugangsdaten im Klartext (INI-Datei) und liegt im Backend-Verzeichnis; dies ist für den geschlossenen Übungs-/Schulungskontext akzeptabel, aber nicht produktionstauglich.
- Kritische Operationen wie /clear-database sind ohne zusätzliche Authentifizierung per POST erreichbar – der Schutz besteht ausschließlich im Frontend-Bestätigungsdialog.

9. Bekannte Probleme & offene Punkte

Diese Liste fasst Auffälligkeiten zusammen, die beim Lesen des Codes aufgefallen sind und für künftige Weiterentwicklung relevant sein könnten.

9.1 Verbindungs-Leck bei Company-Neuanlage

Im except-freien Erfolgspfad von `Company.__init__()` (Neuanlage) wird im finally-Block nur `cursor.close()` aufgerufen; `conn.close()` fehlt. Die ausgeliehene Pool-Verbindung wird dadurch nicht zuverlässig zurückgegeben, was bei häufiger Nutzung (z. B. Massenanlage vieler Unternehmen über die UI) zu Pool-Erschöpfung führen kann.

9.2 Aktien-Symbol-Kollisionen

Zwei unterschiedliche Mechanismen erzeugen Aktienkürzel: `share.py::_generate_unique_symbol` (3 Zeichen, wirft `ShareException` bei Kollision) und `company_import.py::_generate_symbol` (5 Zeichen, keine Kollisionsprüfung). Bei Firmen mit ähnlichem Namensanfang kann Letzteres über INSERT IGNORE stillschweigend zu übersprungenen (nicht importierten) Aktien führen, ohne dass dies im API-Ergebnis sichtbar wird. Eine konsistente Strategie mit automatischem numerischen Suffix bei Kollision (z. B. „APFEL1“, „APFEL2“) wurde als Verbesserung identifiziert, aber noch nicht umgesetzt.

9.3 Blueprint-Registrierung

`blueprints/__init__.py` importiert nur `health_bp`, `imports_bp`, `customers_bp` und `companies_bp` – `shares_bp` und `settings_bp` fehlen dort. Praktisch wirkt sich das nicht aus, da `app.py` alle sechs Blueprints direkt aus den jeweiligen Modulen importiert und registriert, ohne über das `__init__.py`-Paket zu gehen. Für Konsistenz sollte `__init__.py` trotzdem ergänzt werden.

9.4 wirtschaftsbeihilfe ungenutzt

Die Tabelle `wirtschaftsbeihilfe` existiert im Schema, wird aber von keinem Blueprint gelesen oder beschrieben. Vermutlich für eine zukünftige Funktion vorgesehen.

9.5 Saldo-Prüfung nur bei übergebenem Sparbuch-Objekt

Sowohl `CustomerTransaction` als auch `CompanyTransaction` prüfen einen negativen Saldo nur, wenn ihnen ein `customer_savings_book/company_savings_book`-Objekt übergeben wird. Wird die Klasse direkt ohne dieses Objekt verwendet, entfällt die Guthabenprüfung; alle aktuellen Blueprint-Aufrufe übergeben das Objekt jedoch korrekt.

10. Konfiguration & Deployment

10.1 server.config

INI-Datei im Backend-Wurzelverzeichnis, wird von app.py über configparser gelesen:

```
[server] IPAddress = 127.0.0.1 port = 3306 database = stutengarten user = root password = 1234
```

Fehlt die Datei oder der Abschnitt [server], greifen Fallback-Werte (127.0.0.1). „localhost“ wird automatisch auf 127.0.0.1 normalisiert.

10.2 Start der Anwendung (app.py)

Ablauf beim Programmstart:

- 1. Flask-App erzeugen, CORS aktivieren.
- 2. server.config einlesen (IP, Port, DB-Zugangsdaten).
- 3. DbConnector erzeugen und connect() aufrufen → Connection-Pool wird initialisiert.
- 4. Connector unter app.config["DB_CONNECTOR"] ablegen, damit alle Blueprints ihn per current_app.config erreichen.
- 5. Alle sechs Blueprints registrieren (health, imports, customers, companies, settings, shares).
- 6. Waitress-Server auf Port 5000 mit 6 Threads starten (serve(app, host="0.0.0.0", port=5000, threads=6)).
- 7. Im finally-Block wird beim Beenden der Connection-Pool geschlossen.

Ein auskommentierter Flask-Development-Server-Block bleibt im Code als Referenz erhalten, wird aber nicht verwendet – produktiv läuft ausschließlich Waitress.

10.3 Windows-Dienst (NSSM)

Für den produktiven Betrieb auf den Schul-PCs wird app.py nicht manuell gestartet, sondern über NSSM (Non-Sucking Service Manager) als Windows-Dienst registriert. Dadurch startet das Backend automatisch beim Hochfahren des Rechners, ohne dass eine Konsole offen bleiben muss. NSSM wurde gegenüber PyInstaller bevorzugt, da die Anwendung aus mehreren Blueprint-Dateien, externen Konfigurationsdateien (server.config, settings.json) und einer externen MariaDB-Abhängigkeit besteht, was das Bündeln in eine einzelne .exe erschwert.

10.4 Datenbank-Setup

- database.sql – legt alle 11 Tabellen inkl. Indizes an (einmalig auszuführen).
- fill_database.sql – Beispieldaten für manuelle Tests.
- clear_database.sql – identische Funktion wie der Endpunkt POST /clear-database, direkt als SQL-Skript.

11. Zusammenfassung

Das Stutengarten2026-Backend ist eine kompakte, klar in Blueprints und Active-Record-Klassen gegliederte Flask-Anwendung. Die fachliche Trennung in „Kunde“ und „Unternehmen“ zieht sich konsequent durch alle Schichten (eigene Klassen, eigene Exceptions, eigene Blueprints), was den Code gut nachvollziehbar, aber an einigen Stellen redundant macht (CustomerTransaction und CompanyTransaction sind strukturell nahezu identisch). Für Weiterentwicklungen empfiehlt sich, die in Kapitel 9 genannten Punkte – insbesondere das Verbindungs-Leck und die Aktien-Symbol-Kollisionen – zu adressieren, bevor der Funktionsumfang weiter wächst.